

LEARNING TO TEST, TESTING TO LEARN

A Beginner's Daily QA Series

Day 5 - NON-FUNCTIONAL TESTING

NON-FUNCTIONAL TESTING

After functional testing (checking whether features work according to requirements), we test the **non-functional aspects** of an application.

Non-functional testing focuses on:

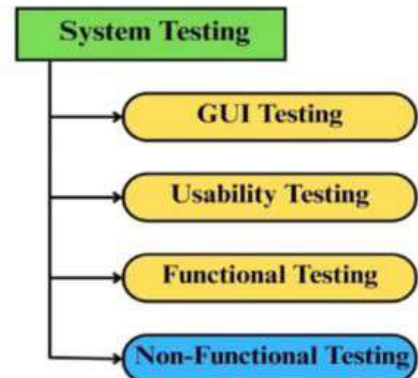
- **Performance:** Speed & responsiveness of the application
- **Security:** Protecting data & preventing unauthorized access
- **Recovery:** Application's ability to recover from failures
- **Installation & Configuration:** Verifying proper setup on different systems
- **Compatibility:** Application works on different devices, OS, browsers
- **Sanitation/Garbage Testing:** Checks for unwanted or leftover data

Key Point:

- Non-functional testing is performed **only after functional testing is stable**.
- It needs **dedicated teams and special environments** (cannot use the same environment as functional testing).
- Requires **special tools and skill sets** (like performance testing engineers, security testing experts).

Functional vs Non-Functional Testing

- **Functional Testing:** Tests “what the application does.” Example: Are all features working as per the customer's requirements?
- **Non-Functional Testing:** Tests “how well the application works.” Example: How fast is it? Is it secure? Does it handle high traffic?



Why is Non-Functional Testing Complex?

- Needs **separate teams** for tasks like performance or security testing.
- Needs **separate environments** for simulating real-world conditions.
- Example: To test 1000 users accessing an app at the same time, we need **virtual users** (dummy users created with tools) instead of physically arranging 1000 people & computers.

★ TYPES OF NON-FUNCTIONAL TESTING ★

1. PERFORMANCE TESTING (5 SUBTYPES)

- Load Testing
- Stress Testing
- Endurance Testing
- Spike Testing
- Volume Testing

2. SECURITY TESTING

3. RECOVERY TESTING

4. COMPATIBILITY TESTING

5. CONFIGURATION TESTING

6. INSTALLATION TESTING

7. SANITATION / GARBAGE TESTING

1. PERFORMANCE TESTING – Overview

- **Performance** generally refers to speed, but it also includes other factors like stability, scalability, and responsiveness.
- To perform performance testing, **special requirement documents** are needed from the client (different from functional testing documents).

Key Details to Collect from the Client Before Performance Testing:

1. **Number of users:** Daily and concurrent users (parallel users).
2. **Usage duration:** How long users stay active on the app (10 hrs/day, 24 hrs/day, etc.).
3. **Peak traffic days:** For example, e-commerce apps have high traffic on sale days.
4. **Database size:** How much data the app needs to handle.
5. **Concurrent usage details:** Approximate number of users accessing at the same time.

1. LOAD TESTING (First Type of Performance Testing)

Load Testing checks how the application performs under the **expected load** (expected number of users or transactions defined by the client).

Key Points:

- Example: If a client expects 1000 concurrent users, we test performance with **exactly 1000 virtual users**, not more or less.
- If a client expects 100 users, we test with 100 users.

What is Measured in Load Testing?

- **Response Time:** Time taken by the server to send back a response after receiving a request.
- **Turnaround Time:** Total time from sending the request to receiving the response (Request Time + Response Time).
- **Throughput:** Number of requests processed by the server **per second**.

Why Use Tools for Load Testing?

- Setting up 1000 physical users and computers is not practical.
- Tools help create **virtual users** that simulate real user traffic.
- Popular Performance Testing Tools:
 - **Apache JMeter** (Open Source – free)
 - **LoadRunner** (Licensed – paid)

Example of Load Testing:

- A new application expects **1000 concurrent users**.
- We create a **load test script** and configure it for 1000 virtual users.
- Run it for **1 hour** and measure:
 - Response time
 - Throughput
 - Error rate, etc.
- After the test, analyze how the application behaves at **expected peak load**.

Key Takeaways:

- Non-functional testing focuses on “how well” an app works (speed, security, recovery).
- Load Testing is the most common performance test, always done with the expected load.
- Response time, turnaround time, and throughput are key metrics measured.

- Tools like JMeter and LoadRunner are essential for simulating users realistically.

2. STRESS TESTING

- **Stress Testing** is a type of **performance testing** where we check how an application performs when the load is **much higher than the expected load**.
- The goal is to test the **application's stability and breaking point** under extreme conditions.

Key Difference Between Load vs Stress Testing

Load Testing:

- Tests the performance under **expected load** (the load given by the client).
- Example: If the client says 1000 users are expected, we test only with 1000 users.
- **Objective:** To verify whether the application can handle the expected user load smoothly.

Stress Testing:

- Tests the performance under **unexpected or beyond the expected load** (higher than the client's expectation).
- Example: If the client says 1000 users, we test with 1200, 1300, or more users.
- **Objective:**
 - To **intentionally break the application** and find its **breaking point** (the point where it stops responding or crashes).
 - To evaluate **how the system behaves under extreme stress**.

How Stress Testing is Performed?

- Start with the expected load (e.g., 1000 users).
- Gradually **increase the number of virtual users** beyond the limit:

- Add 50, 100, or more users step by step.
- Example: Test with 1100 → 1200 → 1300, etc.
- Keep increasing until the application fails to respond properly.
- The **point of failure is the breaking point**, and this is reported to the client.

Example:

If the expected load is 100 users:

- Load Testing → Test only with 100 users.
- Stress Testing → Test with 120, 130, 150 users until the application breaks.

What Metrics Are Measured in Stress Testing?

Stress testing uses the **same parameters** as load testing:

- **Response Time:** Time taken by the server to respond after receiving a request.
- **Turnaround Time:** Request + Response time (total time to complete the operation).
- **Throughput:** Number of requests processed by the server per second.

Additional Focus in Stress Testing:

- **Breaking Point Identification:**
 - Where exactly does the application crash or fail to respond correctly?
 - How much load beyond the expected limit can it handle?

Why is Stress Testing Important?

- Helps identify the **maximum capacity** of the system.
- Ensures that the application won't **suddenly crash** when traffic unexpectedly spikes.

- Provides insights for developers to improve **scalability and stability**.

What Happens If Performance is Poor?

During stress (or any performance) testing, if you notice poor performance (slow response, crashes):

- The issues are reported to the **development team**.
- Developers apply **optimization techniques**:
 - Improve code logic and algorithms (frontend & backend).
 - Optimize **database queries and size**.
 - Enhance **network performance**.

After optimization, performance is re-tested until it meets requirements.

Real-Life Example:

For an application with an expected load of **1000 concurrent users**:

- **Load Test:** Run with 1000 users for 1 hour.
- **Stress Test:** Run with 1200, 1300, or more users.
- Identify at what point (e.g., 1500 users) the application stops responding properly.

Key Takeaways (Load vs Stress Testing)

- **Load Testing Objective:** Can the system handle **expected load**?
- **Stress Testing Objective:** Where is the **breaking point**?
- **Intent in Stress Testing:** *Intentionally push the system until it fails* to understand its limits.
- Both are essential parts of **performance testing** and provide different insights.

3. ENDURANCE TESTING (SOAK TESTING)

- Endurance testing is **a type of Performance Testing**.
- It is also called **Soak Testing**.
- The main goal is to check **if the system can handle the expected load continuously for a long period of time** without performance issues.
- **Key focus areas:**
 - **Time (long duration)**
 - **Stability of the application**

How is it Different from Load and Stress Testing?

Load Testing:

- Tests application with **expected load** (e.g., 100 users)
- Focus is on **how the application performs** under normal expected usage.
- **Time is not the main factor**; it's about performance under expected user load.

Stress Testing:

- Tests application by **going beyond the expected load** (e.g., 1200 users for an app designed for 1000 users).
- The goal is to **break the application** and find its breaking point.

Endurance Testing:

- Tests application with **expected load only** (we do **not** go beyond the expected load).
- **Time is the main factor**.
- We check if the application remains **stable and performs well over an extended duration** (e.g., half a day, 1 day, 2 days, a week, or even a month).

Why is Endurance Testing Important?

- Over a long duration, some applications may face:
 - **Memory leaks** (slowly consuming more memory until failure)
 - **Performance degradation** (slowing down over time)
 - **Stability issues**
- Endurance testing helps **detect these long-term issues early**.

How is Endurance Testing Performed?

1. Set up the expected load:

- Example: Customer says 100 users will use the application at the same time.
- We test with **exactly 100 users** (whatever the customer's expected load is).

2. Run the test for an extended time period:

- Example: Run the system continuously for **24 hours, 2 days, or more** depending on requirements.

3. Monitor application performance:

- Check **stability** (Is the app still working fine after long hours?)
- Check **memory utilization** (Is memory leaking or increasing abnormally?)
- Check **response time** (Is the app slowing down?)

4. Compare results with expected performance:

- If performance matches expectations → **System is stable**.
- If performance degrades → **Report issues to developers**.

5. Optimization : Developers apply fixes and optimizations to improve stability and performance if issues are found.

Example of Endurance Testing

Income Tax Filing Application:

- Used by thousands of users **continuously for long hours/days**.
- **Memory management is critical** to avoid crashes.
- Run test for **24 hours to 2 days** and monitor memory and stability.

Key Points to Remember

- **Endurance testing = Stability testing over time.**
- **We do NOT exceed expected load.**
- **We focus on:**
 - **Time duration**
 - **Memory leaks**
 - **Long-term stability**

4. SPIKE TESTING

Spike Testing is a type of performance testing.

It is used to **analyze the behavior of an application when the number of users suddenly increases or decreases.**

The main purpose is to check:

1. **How the application responds** to an unexpected, sudden load.
2. **Whether it can recover properly** after the sudden spike in traffic.

Key Characteristics of Spike Testing

1. Sudden Load Increase

- Unlike stress testing (where load increases gradually), in spike testing the **load increases abruptly**.
- Example: Load jumps from 100 users to 200 users suddenly.

2. Sudden Load Decrease:

- The load is also **suddenly reduced back** to normal levels (e.g., from 200 users back to 100 or even 80).
- This pattern is sometimes called a **zig-zag load pattern** (increase and decrease suddenly)

3. Beyond Expected Load:

- Spike testing usually tests the system **beyond its normal or expected traffic levels** to see if it breaks or recovers.

Why is Spike Testing Important?

- To **evaluate the stability and recovery capability** of an application when unexpected user traffic occurs.
- To ensure the application:
 - Does not crash suddenly.
 - Handles unexpected spikes efficiently.
 - Returns to normal operation smoothly after the spike.

Example Scenario

Consider an **e-commerce application** like Amazon or Flipkart:

- During **special sales or advertisement campaigns**, huge discounts attract many users.
- The number of users **suddenly increases within a very short time**.
- Spike testing checks:
 - Can the application handle this sudden spike?
 - Does it remain stable during and after the sudden traffic surge?

Difference Between Spike Testing and Stress Testing

Stress Testing:

- Load is increased **gradually** (e.g., 100 → 120 → 150 → 200).
- Aim: To find the breaking point of the system.

Spike Testing:

- Load is increased and decreased **suddenly and unexpectedly** (e.g., 100 → 200 → 90 → 150).
- To test how the system behaves and recovers under **sudden changes** in load.

Main Focus of Spike Testing

- **Performance under sudden load surge.**
- **Application stability and recovery.**
- Identifying:
 - If the system breaks under sudden traffic.
 - Any bottlenecks or performance issues during spike conditions

Summary

Spike testing analyzes **how well an application performs when the load suddenly increases beyond expected limits** and whether it **recovers properly afterward**.

- It is especially important for applications that can face **sudden traffic bursts** (e.g., e-commerce sites, ticket booking systems).

5. VOLUME TESTING

Volume Testing checks how the application performs when it has to handle a **large amount of data at once**.

It focuses on:

- How the application processes, stores, and retrieves data.
- Whether the database and application remain stable with **high data volumes**.
- Identifying any **performance bottlenecks** caused by data growth.

Purpose of Volume Testing – Why is it Important?

- Ensures that:
 - The system can **handle growth of data** over time.
 - The **response time** does not degrade when data increases.
 - There are **no crashes, memory issues, or slow queries**.
 - The **database design and indexing** can support high data loads.
- Especially important for:
 - Banking apps (millions of transactions).
 - E-commerce platforms (huge product & order records).
 - Social media platforms (billions of posts).

How is Volume Testing Performed?

Volume testing can be done in two main ways:

1. Inserting a huge amount of data into the database.

- Example: Add **millions of rows** into user/order tables.

2. Providing the application with a large file or data for processing.

- Example: Uploading a **file with millions of records** for import.

Steps followed:

1. **Prepare large test data.**
2. **Insert the data** into the application/database.
3. **Execute performance tests** (queries, transactions, reports).
4. **Measure:**
 - Response time (is it slower?).
 - Memory usage (any leaks?).
 - CPU usage.
 - Database query performance.
5. **Identify bottlenecks** and fix them (e.g., indexing, query optimization).

What Bottlenecks Can Volume Testing Detect?

- Slow queries due to **lack of proper indexing**.
- **Memory leaks** or crashes due to insufficient memory.
- **Storage issues** (e.g., database runs out of space).
- **Long backup/restore times**.
- **Data retrieval delays** (e.g., search queries take too long).

Example of Volume Testing (Real-World)

- **Scenario:**
A newly developed e-commerce application.
- **Expected Growth:**
Millions of customers & orders in future.
- **Volume Test Plan:**
 - Insert **millions of rows** into product, customer & order tables.

- Perform typical operations (search products, place orders, generate reports).

- **Goal:**

- Identify any performance issues early.
- Ensure app works well even with **massive data growth**.

COMPARISON OF PERFORMANCE TESTING TYPES

Aspect	Load Testing	Stress Testing	Endurance (Soak) Testing	Spike Testing	Volume Testing
Goal	Check if the system performs well under expected user load .	Find the breaking point of the system by testing beyond normal limits.	Verify long-term stability under continuous expected load.	Check how the system reacts to a sudden surge or drop in traffic .	Test system performance with huge amounts of data
Load Pattern / Method	Gradually increase load (users or transactions) to expected levels.	Apply extreme or unrealistic load beyond expected traffic to find failure points	Keep the load constant (expected level) for a long duration (hours/days).	Apply a short burst of sudden traffic spikes (instant increase/decrease).	Insert or process large datasets or high data transfer rates.
Focus Area	Response time, throughput, stability under expected usage.	Stability under extreme conditions; how system fails and recovers .	Memory leaks, performance degradation over time, resource usage trends.	How quickly the app recovers from sudden spikes ; failure handling.	Database and storage handling; query performance with huge data.

Expected Load	Within normal expected user levels.	Beyond expected load (overload conditions)	Expected load for long continuous duration .	Sudden traffic changes (spikes in short time).	Huge data volume , not just user load.
Example Scenario	E-commerce app with 10,000 daily users – test with 10,000 users.	Push app to 50,000+ users suddenly to see when it crashes.	Run app with 10,000 users continuously for 72 hours .	TV ad for e-commerce app causes instant surge in traffic – test sudden 30,000 users at once.	Insert millions of records into DB and run queries to check performance.

Key Differences Explained (in Simple Words)

- **Load vs Stress:**

Load = Normal usage

Stress = Push beyond normal to **find failure point**.

- **Load vs Endurance:**

Both use expected load, but **Endurance tests long-term effects**, e.g., memory leaks.

- **Spike vs Load/Stress:**

Spike is about **sudden instant change in traffic**, while load/stress are more gradual.

- **Volume vs Others:**

Others focus on **users/traffic**, Volume focuses on **data size in DB**.